DATA ENGINEERING FUNDAMENTALS

Types, NULLs, Dates & Strings

Week 3 · Day 2

The details that make numbers correct: choosing data types, reasoning about NULL, and handling dates and text



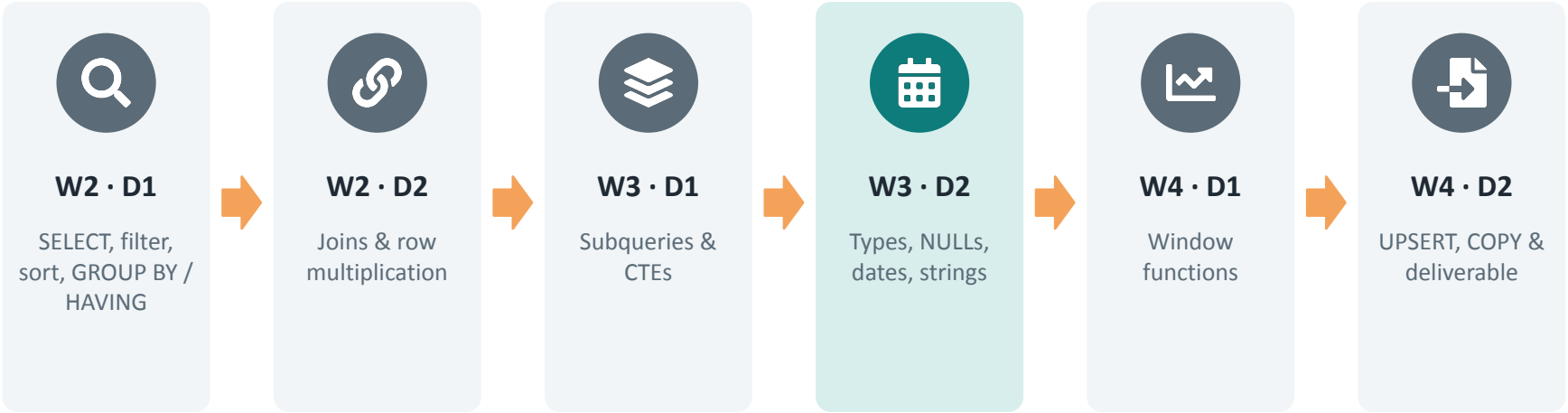
TODAY · ABOUT 2 HOURS 20 MIN

Agenda

25 min	**Data types & casting**	Numeric vs float, ::, ROUND / CEIL / FLOOR
30 min	**NULL semantics**	Three-valued logic, COALESCE, NULLIF, CASE
35 min	**Dates & times**	DATE_TRUNC, parsing, time zones, date spine
20 min	**String handling**	Clean, regex, aggregate, hash text
25 min	**Data-cleaning lab**	Delivery delays, NULL audit, city names
5 min	**Wrap-up & homework**	Query 06

MODULE 2 ROADMAP

Where today fits



Today's output: query 06 — delivery performance (on-time vs late) by state.

01

Data types & casting

The type decides what values are allowed and how math behaves

DATA TYPES

PostgreSQL types you will use most

	Use for	Olist example	Watch out
INTEGER / BIGINT	Counts, IDs that are numbers	payment_installments	7 / 2 = 3 (integer division)
NUMERIC(p, s)	Money — exact decimals	price, payment_value	Slower than float, but exact
DOUBLE PRECISION	Measurements, coordinates	geolocation_lat / lng	0.1 + 0.2 ≠ 0.3 exactly
TEXT / VARCHAR	Names, codes, IDs	order_id, zip_code_prefix	Zip as INTEGER loses leading 0
DATE / TIMESTAMP	Days / date + time	order_purchase_timestamp	Range filters on timestamps
BOOLEAN	Yes/no flags	is_late (we derive it)	Can also be NULL

Rule of thumb: money → NUMERIC, codes that look like numbers (zip, phone) → TEXT, anything you never do math on → TEXT. INTEGER holds whole numbers only. NUMERIC holds exact decimal numbers.

Casting: change a value's type on purpose

```
-- 1. Two ways to cast: timestamp -> date
SELECT order_purchase_timestamp,                -- 2017-10-02 10:56:33
       order_purchase_timestamp::date,          -- 2017-10-02
       CAST(order_purchase_timestamp AS DATE)   -- same result
FROM raw.orders LIMIT 3;



-- 2. A wrong cast loses data
SELECT customer_zip_code_prefix,        -- '01001'
       customer_zip_code_prefix::int    -- 1001  (0 lost) changes TEXT to
INTEGER.
FROM raw.customers
WHERE customer_zip_code_prefix LIKE '0%' LIMIT 3;
```

What happens

- 1. ::date keeps only the day, so you can group orders by day
- 2. Zip codes are text; as a number the leading 0 is gone

Rounding & numeric functions

```
-- Rounding
SELECT ROUND(123.456, 2)   AS r2,       -- 123.46
       ROUND(123.456, -1)  AS r_tens,   -- 120-- -1 means "round to the
nearest ten"
       TRUNC(123.456, 1)   AS t1,       -- 123.4
       CEIL(4.1)           AS up,       -- 5
       FLOOR(4.9)          AS down;     -- 4

-- ROUND(x, n) needs NUMERIC, not float
SELECT ROUND(geolocation_lat::numeric, 3)
FROM raw.geolocation LIMIT 5;

-- Bucketing, clamping, distance, parity
SELECT FLOOR(price / 50) * 50    AS price_band,
       GREATEST(freight_value, 0) AS freight,
       LEAST(price, 1000)        AS capped_price,--"give me the smaller
of price and 1000
       ABS(price - 100)          AS dist_from_100,-- "how far is the
price from 100
       MOD(order_item_id, 2)     AS odd_even -order_item_id/2 (reminder)
FROM raw.order_items LIMIT 5;
```

Which one?

- ROUND: nearest value, n decimals (negative n = tens, hundreds)
- TRUNC: just cut the decimals off
- ROUND goes to the nearest value, and TRUNC simply cuts off the extra digits, so it never rounds up.
- CEIL / FLOOR: next whole number up / down
- GREATEST / LEAST: clamp values; they skip NULLs

02

NULL semantics

NULL means “unknown” — and unknown spreads through everything it touches

Three-valued logic

NULL is not zero and not an empty string. It means "we don't know". Any comparison with an unknown is itself unknown.

Consequences

- NULL = NULL is unknown, not true
- WHERE keeps rows only when the condition is TRUE
- So rows with NULL silently drop out of filters



TRUE

Row is kept by WHERE / ON / HAVING



FALSE

Row is removed



UNKNOWN

Also removed — the surprise

How NULL behaves in everyday SQL

Comparisons

= and <> with NULL give unknown. Use IS NULL / IS NOT NULL.

Aggregates

SUM, AVG, MIN, MAX skip NULLs. AVG of (10, NULL) is 10, not 5.

COUNT

COUNT(*) counts rows; COUNT(col) skips NULLs. The difference = NULL count.

Arithmetic

5 + NULL is NULL. 'abc' || NULL is NULL (CONCAT() skips NULL).

Sorting

NULLs sort last in ASC, first in DESC. Control with NULLS FIRST / LAST.

GROUP BY

All NULLs fall into one group — you will see a blank group row.

Audit NULLs before you trust a column

```
SELECT
  COUNT(*) - COUNT(order_approved_at)
      AS approved_at_nulls,
  COUNT(*) - COUNT(order_delivered_carrier_date)
      AS carrier_nulls,
  COUNT(*) - COUNT(order_delivered_customer_date)
      AS delivered_nulls
FROM raw.orders;
```

RESULT

column (order_…)	NULLs
approved_at	160
delivered_carrier_date	1,783
delivered_customer_date	2,965

Undelivered and canceled orders have no delivery date. Any “average delivery days” must decide what to do with them.

Tools for handling NULL

```
-- COALESCE: first non-NULL value
SELECT COALESCE(review_comment_title, '(no title)')
FROM raw.order_reviews LIMIT 5;

-- NULLIF: turn a value into NULL
-- (avoid divide-by-zero)
SELECT SUM(price) / NULLIF(COUNT(*), 0)
FROM raw.order_items;
```

When to use which

- COALESCE: default for display or joins
- NULLIF: protect divisions; clean '' to NULL

Don't COALESCE numbers to 0 blindly: unknown ≠ zero.

COALESCE: the first value that isn't NULL

How it works




```sql
-- 1. Default text for display
SELECT COALESCE(review_comment_title, '(no title)') AS title
FROM raw.order_reviews;

-- 2. Orders with no payment row: show 0, not NULL
SELECT o.order_id,
       COALESCE(SUM(p.payment_value), 0) AS paid
FROM raw.orders o
LEFT JOIN raw.order_payments p USING (order_id)
GROUP BY o.order_id;

-- 3. All arguments must be the same type
SELECT COALESCE(product_weight_g::text, 'unknown')
FROM raw.products;
```

Rules

- Reads left to right, returns the first non-NULL
- All NULL? Returns NULL
- Arguments must share one type
- Oracle NVL / DECODE, SQL Server ISNULL → COALESCE

Use 0 only when 'none' really means zero.

CASE & NULL-proof cleaning

```
-- COALESCE takes many arguments: first non-NULL wins
SELECT COALESCE(order_delivered_customer_date,
                order_delivered_carrier_date,
                order_purchase_timestamp) AS last_event
FROM raw.orders LIMIT 5;

-- Blank text -> NULL -> default label
SELECT COALESCE(NULLIF(TRIM(product_category_name), ''),
                'unknown') AS category
FROM raw.products;

-- CASE: handle NULL first, then the ranges
SELECT CASE
         WHEN price IS NULL THEN 'missing'
         WHEN price < 50    THEN 'low'
         WHEN price < 200   THEN 'mid'
         ELSE 'high'
       END AS price_tier FROM raw.order_items;
```

Patterns to reuse

- COALESCE(a, b, c, …): fallback chain; all must be the same type
- NULLIF(TRIM(x), ''): standard 'blank means NULL' clean
- CASE stops at the first WHEN that is TRUE

Test IS NULL first — otherwise NULLs fall into ELSE silently.

CASE WHEN inside WHERE

```
-- 1. Different threshold per group
SELECT oi.order_id, c.customer_state, oi.freight_value
FROM raw.order_items oi
JOIN raw.orders o    USING (order_id)
JOIN raw.customers c USING (customer_id)
WHERE oi.freight_value >
      CASE WHEN c.customer_state IN ('SP', 'RJ') THEN 20
           ELSE 40
      END;

-- 2. Same filter with AND / OR (often clearer)
WHERE (c.customer_state IN ('SP', 'RJ') AND oi.freight_value > 20)
   OR (c.customer_state NOT IN ('SP', 'RJ') AND oi.freight_value > 40)

-- 3. Trap: no ELSE -> NULL -> row is dropped
WHERE CASE WHEN order_status = 'delivered' THEN TRUE END
```

How it works

- CASE returns a value; WHERE compares with it
- Useful for a different rule per group
- AND / OR is usually easier to read and faster
- No ELSE gives NULL, and WHERE drops NULL rows

On big tables prefer AND / OR — CASE can stop index use.

03

Dates & times

Truncate, extract, subtract — and filter ranges safely

The date functions you need

```
SELECT order_purchase_timestamp            AS ts,
       order_purchase_timestamp::date      AS day,
       DATE_TRUNC('month', order_purchase_timestamp)
                                           AS month,
       EXTRACT(YEAR FROM order_purchase_timestamp)
                                           AS yr,
       EXTRACT(DOW FROM order_purchase_timestamp)
                                           AS weekday, -- 0=Sun
       TO_CHAR(order_purchase_timestamp, 'YYYY-MM')
                                           AS ym_text,
       order_purchase_timestamp
         + INTERVAL '30 days'              AS plus_30d
FROM raw.orders LIMIT 5;
```

Pick the right one

- Group by month: DATE_TRUNC (still a timestamp, sorts correctly)
- Labels: TO_CHAR (text)
- Parts: EXTRACT

Today, parsing text & durations

```
-- Current date and time
SELECT CURRENT_DATE,      -- today's date
       NOW();             -- timestamptz, start of transaction

-- Text -> date / timestamp using a pattern
SELECT TO_DATE('15/03/2018', 'DD/MM/YYYY'),
       TO_TIMESTAMP('2018-03-15 14:05',
                    'YYYY-MM-DD HH24:MI'),
       MAKE_DATE(2018, 3, 15);

-- Durations-means "the total number of seconds (EPOCH)
SELECT AGE(order_delivered_customer_date,--how much time passed from
           order_purchase_timestamp)      AS age,
       EXTRACT(EPOCH FROM
         order_delivered_customer_date
         - order_purchase_timestamp) / 3600 AS hours
FROM raw.orders LIMIT 5;
```

Pick the right one

- CURRENT_DATE / NOW(): 'last 7 days' filters and loaded_at columns
- TO_DATE / TO_TIMESTAMP: parse text in a known format
- AGE: readable '1 mon 3 days'
- EPOCH / 3600: exact hours for SLAs

NOW() is fixed for the whole transaction — every row gets the same time.

Time zones & date spines

```
-- TIMESTAMP (no zone) -> TIMESTAMPTZ -> another zone
SELECT order_purchase_timestamp
         AT TIME ZONE 'America/Sao_Paulo' AS tz_aware,
       (order_purchase_timestamp
         AT TIME ZONE 'America/Sao_Paulo')
         AT TIME ZONE 'Asia/Bangkok'      AS bangkok_time
FROM raw.orders LIMIT 5;

-- Date spine: one row per day, even with 0 orders
SELECT d::date            AS day,
       COUNT(o.order_id)  AS orders
FROM GENERATE_SERIES('2018-01-01'::timestamp,
                     '2018-01-31'::timestamp,
                     INTERVAL '1 day') AS d
LEFT JOIN raw.orders o
  ON  o.order_purchase_timestamp >= d
  AND o.order_purchase_timestamp <  d + INTERVAL '1 day'
GROUP BY d ORDER BY d;
```

Why it matters

- AT TIME ZONE on TIMESTAMP attaches a zone; on TIMESTAMPTZ it converts
- Own pipelines: store TIMESTAMPTZ (UTC)
- Spine + LEFT JOIN shows empty days; count o.order_id, not *

A daily report with missing days is a silent bug — join to a spine.

Filtering a date range on a timestamp

BETWEEN — misses a day

```
WHERE order_purchase_timestamp
  BETWEEN '2018-01-01'
      AND '2018-01-31'
-- '2018-01-31' means
-- 2018-01-31 00:00:00
```

- Orders on 31 Jan after midnight are excluded

Half-open range — correct

```
WHERE order_purchase_timestamp
        >= '2018-01-01'
  AND order_purchase_timestamp
        <  '2018-02-01'
```

- Includes the whole of January
- Works for any precision

Subtracting timestamps gives an interval; subtracting dates gives whole days: delivered::date - purchased::date = 8

Delivery performance: days and on-time flag

```
WITH delivered AS (
  SELECT o.order_id, c.customer_state,
         o.order_delivered_customer_date::date
           - o.order_purchase_timestamp::date AS days,
         o.order_delivered_customer_date
           > o.order_estimated_delivery_date  AS is_late
  FROM raw.orders o
  JOIN raw.customers c USING (customer_id)
  WHERE o.order_status = 'delivered'
    AND o.order_delivered_customer_date IS NOT NULL
)
SELECT customer_state,
       COUNT(*)                  AS orders,
       ROUND(AVG(days), 1)       AS avg_days,
       ROUND(100.0 * AVG(is_late::int), 1) AS pct_late
FROM delivered
GROUP BY customer_state ORDER BY pct_late DESC;
```

Techniques used

- date − date = days
- A comparison is a BOOLEAN
- AVG(bool::int) = share of TRUE
- IS NOT NULL removes undelivered rows explicitly

This becomes deliverable query 06.

DATES

Date & time cheat sheet

I want to…	Function	Example → result
Get today / now	`CURRENT_DATE · NOW()`	`loaded_at DEFAULT NOW()`
Parse text	`TO_DATE · TO_TIMESTAMP`	`TO_DATE('15/03/2018','DD/MM/YYYY')`
Build from parts	`MAKE_DATE`	`MAKE_DATE(2018, 3, 15)`
Round down to a period	`DATE_TRUNC`	`DATE_TRUNC('month', ts) → 2018-03-01`
Get one part	`EXTRACT · DATE_PART`	`EXTRACT(ISODOW FROM ts) → 1 = Mon`
Format as text	`TO_CHAR`	`TO_CHAR(ts, 'YYYY-MM') → '2018-03'`
Shift a date	`+ / − INTERVAL`	`ts + INTERVAL '30 days'`
Days between two dates	`date − date`	`d2::date − d1::date → 8`
Exact duration	`EXTRACT(EPOCH …)`	`EXTRACT(EPOCH FROM t2 − t1) / 3600`
Readable duration	`AGE`	`AGE(t2, t1) → 1 mon 3 days`
Change time zone	`AT TIME ZONE`	`ts AT TIME ZONE 'UTC'`
List every day in a range	`GENERATE_SERIES`	`(d1, d2, INTERVAL '1 day')`

Date functions on the orders table

All rows: `FROM raw.orders WHERE order_id = 'e481f51cbdc54678b7cc49136f2d6af7'` (purchased 2017-10-02 10:56:33)

Function	Example on raw.orders	Result
CURRENT_DATE	`WHERE order_purchase_timestamp <= CURRENT_DATE - 30`	`0 rows (ends 2018)`
TO_TIMESTAMP	`TO_TIMESTAMP(order_purchase_timestamp::text, 'YYYY-MM-DD HH24:MI:SS')`	`text -> timestamp`
MAKE_DATE	`WHERE order_purchase_timestamp >= MAKE_DATE(2017, 10, 1)`	`order is kept`
DATE_TRUNC	`DATE_TRUNC('month', order_purchase_timestamp)`	`2017-10-01 00:00:00`
EXTRACT	`EXTRACT(ISODOW FROM order_purchase_timestamp)`	`1 (Monday)`
TO_CHAR	`TO_CHAR(order_purchase_timestamp, 'YYYY-MM')`	`'2017-10'`
+ INTERVAL	`order_purchase_timestamp + INTERVAL '30 days'`	`2017-11-01 10:56:33`
date − date	`order_delivered_customer_date::date - order_purchase_timestamp::date`	`8 (days)`
EXTRACT(EPOCH)	`EXTRACT(EPOCH FROM order_approved_at - order_purchase_timestamp) / 60`	`10.7 (minutes)`
AGE	`AGE(order_delivered_customer_date, order_purchase_timestamp)`	`8 days 10:28:40`
AT TIME ZONE	`order_purchase_timestamp AT TIME ZONE 'America/Sao_Paulo' AT TIME ZONE 'Asia/Bangkok'`	`2017-10-02 20:56:33`
GENERATE_SERIES	`GENERATE_SERIES(order_purchase_timestamp::date, order_delivered_customer_date::date, INTERVAL '1 day')`	`9 rows: 2-10 Oct`

04

String handling

Clean, split and search text columns

String functions for cleaning data

```
SELECT customer_city,
       INITCAP(customer_city)          AS nice_city,
       UPPER(TRIM(customer_state))     AS state,
       LENGTH(customer_zip_code_prefix) AS zip_len,
       LEFT(customer_zip_code_prefix, 2) AS zip_region,
       customer_city || ' - ' || customer_state
                                       AS label
FROM raw.customers LIMIT 5;

SELECT SPLIT_PART('health_beauty', '_', 1);  -- health
SELECT REPLACE('bed_bath_table', '_', ' ');

-- Case-insensitive search
SELECT COUNT(*) FROM raw.customers
WHERE customer_city ILIKE '%paulo%';
```

Cleaning checklist

- TRIM spaces before comparing
- Normalise case with LOWER / UPPER
- || returns NULL if any part is NULL — CONCAT() doesn't
- ILIKE = case-insensitive LIKE

Padding, slicing & regex

```
-- Pad, slice, locate
SELECT LPAD('1001', 5, '0')             AS zip,  -- 01001
       SUBSTRING('health_beauty', 1, 6) AS part, -- health
       STRPOS('bed_bath', '_')          AS pos,  -- 4
       RIGHT('abc123', 3)               AS tail; -- 123

-- Regex replace: keep digits only
SELECT REGEXP_REPLACE('(11) 9876-5432',
                      '[^0-9]', '', 'g');  -- 1198765432

-- Regex test: find invalid zip prefixes
SELECT customer_zip_code_prefix
FROM raw.customers
WHERE customer_zip_code_prefix !~ '^[0-9]{5}$';

■  Regex extract: first number in a text
SUBSTRING( text , start , length )
SELECT SUBSTRING(order_id, 1, 8) AS short_id, -- 'e481f51c'
SUBSTRING(order_purchase_timestamp::text, 1, 7) AS year_month --
'2017-10'
FROM raw.orders LIMIT 3;
```

Cleaning recipes

- LPAD: restore leading zeros lost in Excel / CSV
- ~ matches, !~ does not, ~* ignores case
- Regex tests make good data-quality checks

Without 'g', REGEXP_REPLACE only replaces the first match.

Combine, split & fingerprint text

```
-- Remove accents: são paulo -> sao paulo
SELECT TRANSLATE('são paulo',
                 'áàâãéêíóôõúç', 'aaaaeeioooouc');
TRANSLATE( text , from_chars , to_chars )

-- Join with a separator, skipping NULLs
SELECT CONCAT_WS(' - ', 'Sao Paulo', NULL, 'SP');

-- Many rows -> one string
SELECT order_id,
       STRING_AGG(product_id, ', ' ORDER BY order_item_id)
FROM raw.order_items GROUP BY order_id LIMIT 5;

-- One string -> many rows text → array, array → many rows
SELECT UNNEST(STRING_TO_ARRAY('a,b,c', ','));

-- Row fingerprint for change detection

SELECT customer_id,--row(packs several column values into one value)
       MD5(ROW(customer_city, customer_state)::text) AS h
FROM raw.customers LIMIT 5;--fixed 32-character code called a hash
```

Data-engineering uses

- TRANSLATE: match city names across tables
- CONCAT_WS: labels and keys without NULL gaps
- STRING_AGG: one row per order for reports
- MD5 row hash: did this row change?

Hash ROW(…)::text, not CONCAT_WS — it drops NULLs, so (a, NULL) = (NULL, a).

STRINGS

Joining text: ||, CONCAT & CONCAT_WS

```
-- Same values, one NULL in the middle
SELECT 'Sao Paulo' || ' - ' || NULL || ' - ' || 'SP'   AS with_pipes,      -- NULL
       CONCAT('Sao Paulo', ' - ', NULL, ' - ', 'SP')     AS with_concat,     -- 'Sao Paulo -  - SP'
       CONCAT_WS(' - ', 'Sao Paulo', NULL, 'SP')         AS with_concat_ws;  -- 'Sao Paulo - SP'

-- Olist: no NULLs here, so all three give 'franca - SP'
SELECT customer_city || ' - ' || customer_state,
       CONCAT(customer_city, ' - ', customer_state),
       CONCAT_WS(' - ', customer_city, customer_state)
FROM raw.customers LIMIT 3;
```

Method	Result	With NULL	Use when
`a || b || c`	`NULL`	one NULL makes the whole result NULL	columns never NULL; shortest
`CONCAT(a, b, c)`	`'Sao Paulo -  - SP'`	NULL = empty, but both separators stay	NULLs possible, no separator
`CONCAT_WS(sep, a, b, c)`	`'Sao Paulo - SP'`	skips NULL and its separator	same separator, NULLs possible

Remember: || breaks on NULL · CONCAT ignores NULL · CONCAT_WS ignores NULL and its separator. For hash keys use ROW(…)::text.

REGEXP_REPLACE: arguments & patterns

```
REGEXP_REPLACE( source, pattern, replacement, flags )
```

Arguments (read by position)

#	Argument	Example
1	source — the text or column	`customer_city`
2	pattern — what to find	`'_'`
3	replacement — what to put in	`' '`
4	flags (optional)	`'g'`

Flags: 'g' = every match · 'i' = ignore case · 'gi' = both

Trap: REGEXP_REPLACE('a_b_c', '_', 'g') → 'agb_c'. Only 3 arguments, so 'g' became the replacement.

Useful patterns

Pattern	Means
`[0-9]` or `\d`	one digit
`[^0-9]` or `\D`	anything that is NOT a digit
`[a-z]`	one lowercase letter
`[^a-z ]`	NOT a letter and NOT a space
`\s+`	one or more spaces
`[\r\n]+`	one or more line breaks
`^` `$`	start of text · end of text
`.` `\.`	any character · a real dot
`{5}` `{2,}`	exactly 5 · 2 or more
`( )` `\1`	group · reuse group 1

Useful patterns on Olist data

Goal	Example	Result
Line breaks → space	`REGEXP_REPLACE(review_comment_message, '[\r\n]+', ' ', 'g')`	`text on one line`
Underscores → space	`REGEXP_REPLACE(product_category_name, '_', ' ', 'g')`	`'cama mesa banho'`
Many spaces → one	`REGEXP_REPLACE(review_comment_message, '\s{2,}', ' ', 'g')`	`'muito bom'`
Remove symbols	`REGEXP_REPLACE(customer_city, '[^a-z ]', '', 'g')`	`"d'oeste" → 'doeste'`
First word only	`REGEXP_REPLACE(product_category_name, '_.*$', '')`	`'cama'`
Remove leading zeros	`REGEXP_REPLACE(customer_zip_code_prefix, '^0+', '')`	`'01001' → '1001'`
Short ID	`REGEXP_REPLACE(order_id, '^(.{8}).*$', '\1')`	`'e481f51c'`
Reorder date text	`REGEXP_REPLACE(order_purchase_timestamp::text, '^(\d{4})-(\d{2})-(\d{2}).*$', '\3/\2/\1')`	`'02/10/2017'`
Ignore case ('i')	`REGEXP_REPLACE(customer_state, '^sp$', 'Sao Paulo', 'i')`	`'SP' → 'Sao Paulo'`

Tip: REGEXP_REPLACE only changes what you SELECT — the table stays the same until you UPDATE it.

REGEXP_REPLACE in SELECT and WHERE

```
-- SELECT: line breaks in review text -> one space
SELECT review_comment_message,
       REGEXP_REPLACE(review_comment_message,
                      '[\r\n]+', ' ', 'g') AS one_line
FROM raw.order_reviews
WHERE review_comment_message ~ '[\r\n]';

-- SELECT: underscores -> spaces
SELECT product_category_name,
       REGEXP_REPLACE(product_category_name, '_', ' ', 'g')
FROM raw.products;          -- cama_mesa_banho -> cama mesa banho

-- WHERE: find cities with symbols (not a-z or space)
SELECT DISTINCT customer_city
FROM raw.customers
WHERE REGEXP_REPLACE(customer_city, '[^a-z ]', '', 'g')
      <> customer_city;     -- santa barbara d'oeste
```

How to read it

- REGEXP_REPLACE(text, pattern, new_text, 'g')
- [\r\n]+: one or more line breaks
- [^a-z]: anything that is not a letter or space
- 'g': replace every match, not just the first

In WHERE it runs on every row — clean once at load time.

05

Data-cleaning lab

Audit, clean and measure

TROUBLESHOOTING

Common type, NULL & date errors

`Percentage shows 0`

Integer division. Use 100.0 * x / y or cast to numeric.

`WHERE col <> 'x' drops rows`

Rows where col is NULL are unknown, not true. Add OR col IS NULL.

`invalid input syntax for type date`

A text value can't be cast. Find bad rows first; clean or NULLIF them.

`Months sorted Apr, Aug, Dec…`

You sorted TO_CHAR text. Sort by DATE_TRUNC('month', ts) instead.

BEFORE WEEK 4

Wrap-up & homework



Query 06

sql/06_delivery_performance.sql — avg days and % late by state.



NULL audit notes

Add a comment in each query explaining how NULLs are handled.



Commit & push

You should now have queries 01–06 in sql/.

Week 4 · Day 1: window functions — ROW_NUMBER, RANK, LAG/LEAD, running totals